

UNIT III COMPUTER FUNDAMENTALS

Functional Units of a Digital Computer: Von Neumann Architecture – Operation and Operands of Computer Hardware Instruction – Instruction Set Architecture (ISA): Memory Location, Address and Operation – Instruction and Instruction Sequencing – Addressing Modes, Encoding of Machine Instruction – Interaction between Assembly and High Level Language.

- ❖ **Explain the components of a computer system with the block diagram in detail (Apr/May 21)**

Components of a computer system

A computer consists of five functionally independent main parts. They are

1. Input
2. Memory
3. Arithmetic and logic unit
4. Output
5. Control unit

Basic functional units of a computer

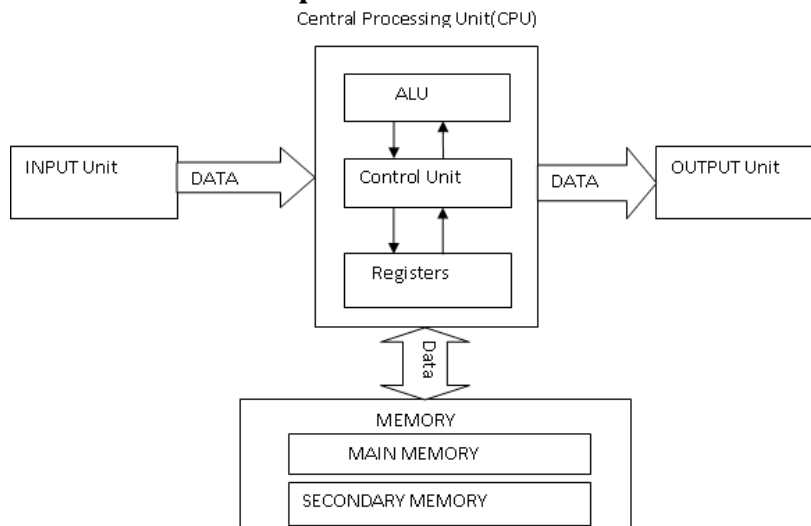


Fig 3.1 The basic components of computer system

The computer accepts programs and the data through an input and stores them in the memory. The stored data are processed by the arithmetic and logic unit under program control. The processed data is delivered through the output unit. All above activities are directed by control unit.

a. Input unit

The computer accepts coded information through input unit. The input can be from human operators; electromechanical devices such as keyboards or from other computer over communication lines.

Examples of input devices are Keyboard, joysticks; trackballs and mouse are used as graphic input devices in conjunction with display.

- It is a common input device.
- Whenever a key is pressed; the corresponding letter or digit is automatically translated into its corresponding binary code and transmitted over cable to the memory of the computer.

b. Memory unit

Memory unit is used to store programs as well as data. Memory is classified into primary and secondary storage.

Primary Storage

It also called main memory. It operates at high speed and it is expensive. It is made up of large number of semiconductor storage cells, each capable of storing one bit of information. These cells are grouped together in a fixed size called word. This facilitates reading and writing the content of one word (n bits) in single basic operation instead of reading and writing one bit for each operation.

Secondary Storage

It is slow in speed. It is cheaper than primary memory. Its capacity is high. It is used to store information that is not accessed frequently. Various secondary devices are magnetic tapes and disks, optical disks (CD-ROMs), floppy etc.

c. Arithmetic and Logic Unit

Arithmetic and logic unit (ALU) and control unit together form a processor. Actual execution of most computer operations takes place in arithmetic and logic unit of the processor. Example: Suppose two numbers located in the memory are to be added. They are brought into the processor, and the actual addition is carried out by the ALU.

Registers are high speed storage elements available in the processor. Each register can store one word of data. When operands are brought into the processor for any operation, they are stored in the registers. Accessing data from register is faster than that of the memory.

d. Output unit

The function of output unit is to produce processed result to the outside world in human understandable form. Examples of output devices are Graphical display, Printers such as inkjet, laser, dot matrix and so on. The laser printer works faster.

e. Control unit

Control unit coordinates the operation of memory, arithmetic and logic unit, input unit, and output unit in some proper way. The **control unit** issues **control signals** that cause the CPU (and other components of the computer) to fetch the instruction to the IR (Instruction Register) and then execute the actions dictated by the machine language instruction that has been stored there. Control units are well defined, physically separate unit that interact with other parts of the machine. A set of control lines carries the signals used for timing and synchronization of events in all units. Example: Data transfers between the processor and the memory are controlled by the control unit through timing signals. **Timing signals** are the signals that determine when a given action is to take place.

Basic Operational Concept

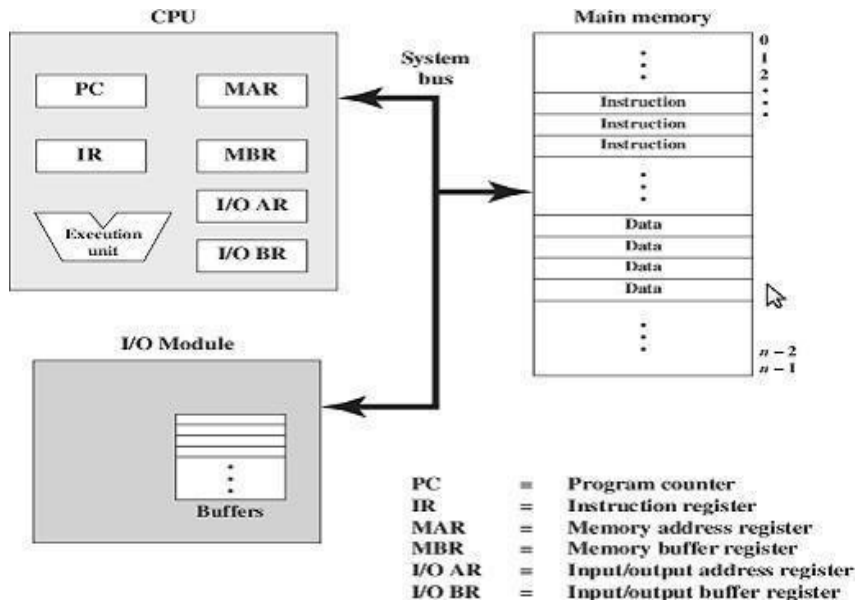


Fig 3.2 Computer Components

Computer Components:

Top-Level view

PC the **program counter** contains the address of the assembly language instruction to be executed next. IR the **instruction register** contains the binary word corresponding to the machine language version of the instruction currently being executed.

MAR the **memory address register** contains the address of the word in main memory that is being accessed. The word being addressed contains either data or a machine language instruction to be executed.

MBR the **memory buffer register** (also called MDR for memory data register) is the register used to communicate data to and from the memory.

The operation of a processor is characterized by a fetch-decode-execute cycle. In the first phase of the cycle, the processor fetches an instruction from memory. The address of the instruction to fetch is stored in an internal register named the program counter, or PC. As the processor is waiting for the memory to respond with the instruction, it increments the PC. This means the fetch phase of the next cycle will fetch the instruction in the next sequential location in memory. In the decode phase the processor stores the information returned by the memory in another internal register, known as the instruction register, or IR. The IR now holds a

single machine instruction encoded as a binary number. The processor decodes the value in the IR in order to figure out which operations to perform in the next stage.

In the execution stage the processor actually carries out the instruction. This step often requires further memory operations; for example, the instruction may direct the processor to fetch two operands from memory, add them, and store the result in a third location (the addresses of the operands and the result are also encoded as part of the instruction). At the end of this phase the machine starts the cycle over again by entering the fetch phase for the next instruction. The CPU exchanges data with memory. For this purpose, it typically makes use of **two internal (to the CPU) register**:

- A memory address register (MAR), which specifies the address in memory for the next read or write, and
- A memory buffer register (MBR), which contains the data to be written into memory or receives the data read from memory.

An I/O address register (I/OAR) specifies a particular I/O device.

An I/O buffer (I/OBR) register is used for the exchange of data between an I/O module and the CPU. A memory module consists of a set of locations, defined by sequentially numbered addresses. Each location contains a binary number that can be interpreted as either an instruction or data. An I/O module transfers data from external devices to CPU and memory, and vice versa.

It contains internal buffers for temporarily holding these data until they can be sent on. Instructions can be classified as one of three major types: arithmetic/logic, data transfer, and control. Arithmetic and logic instructions apply primitive functions of one or two arguments, for example addition, multiplication, or logical AND.

Von Neumann Architecture

Von Neumann architecture was first published by John von Neumann in 1945. His computer architecture design consists of a Control Unit, Arithmetic and Logic Unit (ALU), Memory Unit, Registers and Inputs/Outputs.

Von Neumann architecture is based on the stored-program computer concept, where instruction data and program data are stored in the same memory. This design is still used in most computers produced today.

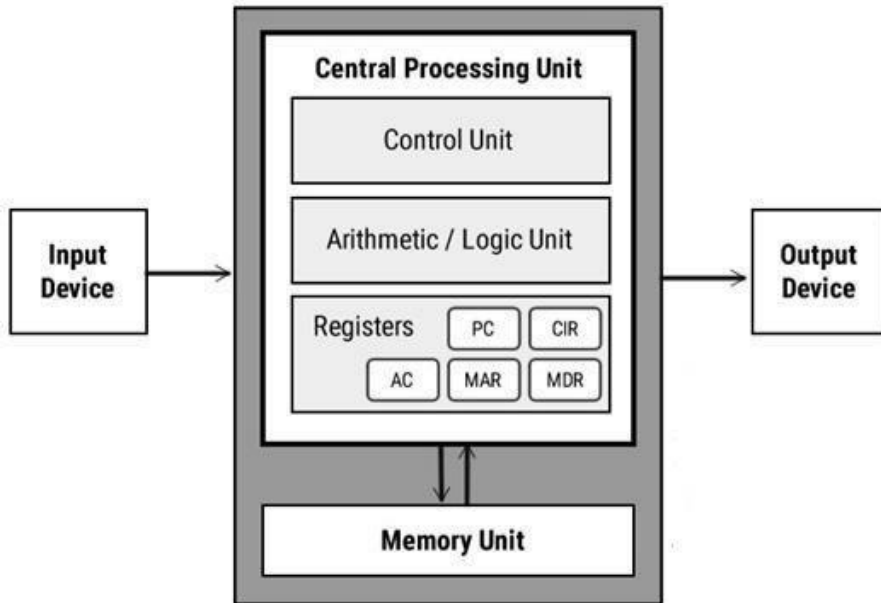


Fig 3.3 Von Neumann architecture

Central Processing Unit (CPU)

The Central Processing Unit (CPU) is the electronic circuit responsible for executing the instructions of a computer program. It is sometimes referred to as the microprocessor or processor. The CPU contains the ALU, CU and a variety of registers.

Registers

Registers are high speed storage areas in the CPU. All data must be stored in a register before it can be processed.

MAR	Memory Address Register	Holds the memory location of data that needs to be accessed
MDR	Memory Data Register	Holds data that is being transferred to or from memory
AC	Accumulator	Where intermediate arithmetic and logic results are stored
PC	Program Counter	Contains the address of the next instruction to be executed
CIR	Current Instruction Register	Contains the current instruction during processing

Arithmetic and Logic Unit (ALU)

The ALU allows arithmetic (add, subtract etc.) and logic (AND, OR, NOT etc.) operations to be carried out.

Control Unit (CU)

The control unit controls the operation of the computer's ALU, memory and input/output devices, telling them how to respond to the program instructions it has just read and interpreted from the memory unit. The control unit also provides the timing and control signals required by other computer components.

Buses

Buses are the means by which data is transmitted from one part of a computer to another, connecting all major internal components to the CPU and memory.

A standard CPU system bus is comprised of a control bus, data bus and address bus.

Address Bus	Carries the addresses of data (but not the data) between the processor and memory
Data Bus	Carries the data from input/output devices
Control Bus	Carries control signals/commands from the CPU (and status signals from other devices) in order to control and coordinate all the activities within the computer

Memory Unit

The Memory unit consists of RAM, sometimes referred to as primary or main memory. Unlike a hard drive(secondary memory), this memory is fast and also directly accessible by the CPU. RAM is split into partitions. Each partition consists of an address and its contents (both in binary form). The address will uniquely identify every location in the memory. Loading data from permanent memory (hard drive), into the faster and directly accessible temporary memory (RAM), allows the CPU to operate much quicker.

Von Neumann bottleneck

Whatever we do to enhance performance, we cannot get away from the fact that instructions can only be done one at a time and can only be carried out sequentially. Both of these factors hold back the competence of the CPU. This is commonly referred to as the 'Von Neumann bottleneck'. We can provide a Von Neumann processor with more cache, more RAM, or faster components but if original gains are to be made in CPU performance then an influential inspection needs to take place of CPU configuration.

- ❖ Explain instruction set Architecture? Give examples (April/May 2016)
- ❖ Explain memory locations and addresses (Nov/Dec 2015).

Instruction format

The instruction format of an instruction is usually depicted in a rectangular box symbolizing the bits of the instruction as they appear in memory words or in a control register. An instruction format defines the layout of the bits of an instruction, in terms of its constituent parts. The bits of an instruction are divided into groups called fields. The most common fields found in instruction formats are:

- An operation code field that specifies the operation to be performed.
- An address field that designates a memory address or a processor register.
- A mode field that specifies the way the operand or the effective address is determined

Opcode	Mode	Address
--------	------	---------

Fig 3.4 Instruction fields

Other special fields are sometimes employed under certain circumstances. The operation code field of an instruction is a group of bits that define various processor operations, such as add, subtract, complement and shift. Address fields contain either a memory address field or a register address. Mode fields offer a variety of ways in which an operand is chosen.

There are mainly four types of instruction formats:

- Three address instructions
- Two address instructions
- One address instructions
- Zero address instructions

Three address instructions

Computers with three address instructions formats can use each address field to specify either a processor register or a memory operand. The program in assembly language that evaluates $X = (A+B)*(C+D)$ is shown below, together with comments that explain the register transfer operation of each instruction.

Add R1, A, B	$R1 \leftarrow M[A] + M[B]$
Add R2, C, D	$R2 \leftarrow M[C] + M[D]$
Mul X, R1, R2	$M[X] \leftarrow R1 * R2$

It is assumed that the computer has two processor registers, R1 and R2. The symbol $M[A]$ denotes the operand at memory address symbolized by A. the advantage of the three address format is that it results in short programs when evaluating arithmetic expressions. The disadvantage is that the binary coded instructions require too many bits to specify three addresses. An example of an commercial computer that uses three address instructions is the Cyber 170. The instruction formats in the Cyber computer are restricted to either three register address fields or two register address fields and one memory address field.

Two address instructions

Two address instructions are the most common in commercial computers. Here again each address field can specify either a processor register or a memory word. The program to evaluate $X = (A+B)*(C+D)$ is as follows:

MOV	R1, A	$R1 \leftarrow M[A]$
ADD	R2, B	$R1 \leftarrow R1 + M[B]$
MOV	R2, C	$R2 \leftarrow M[C]$
ADD	R2, D	$R2 \leftarrow R2 + M[D]$
MUL	R1, R2	$R1 \leftarrow R1 * R2$
MOV	X, R1	$M[X] \leftarrow R1$

The MOV instruction moves or transfers the operands to and from memory and processor registers. The first symbol listed in an instruction is assumed to be both a source and the destination where the result of the operation is transferred.

One address instructions

One address instructions use an implied accumulator (AC) register for all data manipulation. For multiplication and division there is a need for a second register. However, here we will neglect the second register and assume that the AC contains the result of all operations. The program to evaluate $X = (A+B)*(C+D)$ is

LOAD	A	$AC \leftarrow M[A]$
ADD	B	$AC \leftarrow AC + M[B]$
STORE	T	$M[T] \leftarrow AC$
LOAD	C	$AC \leftarrow M[C]$
ADD	D	$AC \leftarrow AC + M[D]$
MUL	T	$AC \leftarrow AC * M[T]$
STORE	X	$M[X] \leftarrow AC$

All operations are done between the AC register and a memory operand. T is the address of a temporary memory location required for storing the intermediate result. Commercially available computers also use this type of instruction format.

Zero address instructions

A stack organized computer does not use an address field for the instructions ADD and MUL. The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack. The following program shows how $X=(A+B)*(C+D)$ will be written for a stack organized computer. (TOS stands for top of stack.)

PUSH	A	$TOS \leftarrow A$
PUSH	B	$TOS \leftarrow B$
ADD		$TOS \leftarrow (A + B)$
PUSH	C	$TOS \leftarrow C$
PUSH	D	$TOS \leftarrow D$
ADD		$TOS \leftarrow (C+D)$
MUL		$TOS \leftarrow (C+D) * (A+B)$
POP	X	$M[X] \leftarrow TOS$

To evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression into reverse polish notation. The name “zero address” is given to this type of computer because of the absence of an address field in computational instructions.

OPERATIONS AND OPERANDS

Every computer must be able to perform arithmetic. The MIPS assembly language notation `add a, b, c` instructs a computer to add the two variables `b` and `c` and to put their sum in `a`. This notation is rigid in that each MIPS arithmetic instruction performs only one operation and must always have exactly three variables. For example, suppose we want to place the sum of four variables `b`, `c`, `d`, and `e` into variable `a`.

The following sequence of instructions adds the four variables:

`add a, b, c` # The sum of `b` and `c` is placed in `a`

`add a, a, d` # The sum of `b`, `c`, and `d` is now in `a`

`add a, a, e` # The sum of `b`, `c`, `d`, and `e` is now in `a`

Category	Instruction	Example	Meaning	Comments
Arithmetic	add	add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	Three operands; data in registers
	subtract	sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	Three operands; data in registers
	add immediate	addi \$s1, \$s2, 100	$\$s1 = \$s2 + 100$	Used to add constants
Data transfer	load word	lw \$s1, 100(\$s2)	$\$s1 = \text{Memory}[\$s2 + 100]$	Word from memory to register
	store word	sw \$s1, 100(\$s2)	$\text{Memory}[\$s2 + 100] = \$s1$	Word from register to memory
	load byte	lb \$s1, 100(\$s2)	$\$s1 = \text{Memory}[\$s2 + 100]$	Byte from memory to register
	store byte	sb \$s1, 100(\$s2)	$\text{Memory}[\$s2 + 100] = \$s1$	Byte from register to memory
	load upper immediate	lui \$s1, 100	$\$s1 = 100 * 2^{16}$	Loads constant in upper 16 bits
Conditional branch	branch on equal	beq \$s1, \$s2, 25	If $(\$s1 == \$s2)$ go to PC + 4 + 100	Equal test; PC-relative branch
	branch on not equal	bne \$s1, \$s2, 25	If $(\$s1 \neq \$s2)$ go to PC + 4 + 100	Not equal test; PC-relative
	set on less than	slt \$s1, \$s2, \$s3	If $(\$s2 < \$s3)$ $\$s1 = 1$; else $\$s1 = 0$	Compare less than; for beq, bne
	set less than immediate	slti \$s1, \$s2, 100	If $(\$s2 < 100)$ $\$s1 = 1$; else $\$s1 = 0$	Compare less than constant
Unconditional jump	jump	j 2500	go to 10000	Jump to target address
	jump register	jr \$ra	go to \$ra	For switch, procedure return
	jump and link	jal 2500	$\$ra = PC + 4$; go to 10000	For procedure call

Fig 3.5 MIPS Instructions

Thus, it takes three instructions to sum the four variables. The words to the right of the sharp symbol (#) on each line above are comments for the human reader, so the computer ignores them.

MIPS operands

Name	Example	Comments
32 registers	$\$s0-\$s7$, $\$t0-\$t9$, \$zero, $\$a0-\$a3$, $\$v0-\$v1$, \$gp, \$fp, \$sp, \$ra, \$at	Fast locations for data. In MIPS, data must be in registers to perform arithmetic. MIPS register \$zero always equals 0. Register \$at is reserved for the assembler to handle large constants.
2^{30} memory words	Memory[0], Memory[4], ..., Memory[4294967292]	Accessed only by data transfer instructions. MIPS uses byte addresses, so sequential words differ by 4. Memory holds data structures, such as arrays, and spilled registers, such as those saved on procedure calls.

Fig. 3.6 MIPS Operands

REPRESENTING INSTRUCTIONS

Numbers are kept in computer hardware as a series of high and low electronic signals, and so they are considered base 2 numbers. (Just as base 10 numbers are called decimal numbers, base 2 numbers are called binary numbers.) A single digit of a binary number is thus the “atom” of computing, since all information is composed of binary digits or bits. This fundamental building block can be one of two values, which can be thought of as several alternatives: high or low, on or off, true or false, or 1 or 0.

Instructions are also kept in the computer as a series of high and low electronic signals and may be represented as numbers. In fact, each piece of an instruction can be considered as an individual number, and placing these numbers side by side forms the instruction.

Since registers are part of almost all instructions, there must be a convention to map register names into numbers. In MIPS assembly language, registers \$s0 to \$s7 map onto registers 16 to 23, and registers \$t0 to \$t7 map onto registers 8 to 15. Hence, \$s0 means register 16, \$s1 means register 17, \$s2 means register 18, . . . , \$t0 means register 8, \$t1 means register 9, and so on.

MIPS Fields

The meaning of each name of the fields in MIPS instructions:

op: Basic operation of the instruction, traditionally called the opcode. rs: The first register source operand.

rt: The second register source operand.

rd: The register destination operand. It gets the result of the operation.

shamt: Shift amount.

funct: Function. This field selects the specific variant of the operation in the op field and is sometimes called the function code.

A problem occurs when an instruction needs longer fields than those specified above. For example, the load word instruction must specify two registers and a constant. If the address were to use one of the 5-bit fields in the format above, the constant within the load word instruction would be limited to only 25 or 32. This constant is used to

select elements from arrays or data structures, and it often needs to be much larger than 32. This 5-bit field is too small to be useful.

Today's computers are built on two key principles:

1. Instructions are represented as numbers.
2. Programs are stored in memory to be read or written, just like numbers.

These principles lead to the stored-program concept; its invention let the computing genie out of its bottle. Specifically, memory can contain the source code for an editor program, the corresponding compiled machine code, the text that the compiled program is using, and even the compiler that generated the machine code. One consequence of instructions as numbers is that programs are often shipped as files of binary numbers. The commercial implication is that computers can inherit ready-made software provided they are compatible with an existing instruction set. Such "binary compatibility" often leads industry to align around a small number of instruction set architectures.

LOGICAL OPERATIONS

Although the first computers operated on full words, it soon became clear that it was useful to operate on fields of bits within a word or even on individual bits. Examining characters within a word, each of which is stored as 8 bits, is one example of such an operation. It follows that operations were added to programming languages and instruction set architectures to simplify, among other things, the packing and unpacking of bits into words. These instructions are called logical operations.

CONTROL OPERATIONS

What distinguishes a computer from a simple calculator is its ability to make decisions. Based on the input data and the values created during computation, different instructions execute. Decision making is commonly represented in programming languages using the if statement, sometimes combined with go to statements and labels. MIPS assembly language includes two decision-making instructions, similar to an if statement with a go to. The first instruction is

beq register1, register2, L1

This instruction means go to the statement labeled L1 if the value in register1 equals the value in register2. The mnemonic beq stands for branch if equal.

The second instruction is

bne register1, register2, L1

It means go to the statement labeled L1 if the value in register1 does not equal the value in register2. The mnemonic bne stands for branch if not equal. These two instructions are traditionally called conditional branches.

Case/Switch statement

Most programming languages have a case or switch statement that allows the programmer to select one of many alternatives depending on a single value. The simplest way to implement switch is via a sequence of conditional tests, turning the switch statement into a chain of if-then-else statements. Sometimes the alternatives may be more efficiently encoded as a table of addresses of alternative instruction sequences, called a jump address table, and the program needs only to index into the table and then jump to the appropriate sequence. The jump table is then just an array of words containing addresses that correspond to labels in the code.

To support such situations, computers like MIPS include a jump register instruction (jr), meaning an unconditional jump to the address specified in a register. The program loads the appropriate entry from the jump table into a register, and then it jumps to the proper address using a jump register.

Instruction Set Architecture

A complete instruction set, including operand addressing methods, is often referred to as the *instruction set architecture (ISA)* of a processor.

The vast majority of programs are written in high-level languages such as C, C++, or Java. To execute a high-level language program on a processor, the program must be translated into the machine language for that processor, which is done by a compiler program. Assembly language is a readable symbolic representation of machine language.

Memory Locations and Addresses

The memory consists of many millions of storage *cells*, each of which can store a *bit* of information having the value 0 or 1. Because a single bit represents a very small amount of information, bits are seldom handled individually. The usual approach is to deal with them in groups of fixed size. For this purpose, the memory is organized so that a group of n bits can be stored or retrieved in a single, basic operation. Each group of n bits is referred to as a *word* of information, and n is called the *word length*. The memory of a computer can be schematically represented as a collection of words, as shown in Fig 3.7. Modern computers have word lengths that typically range from 16 to 64 bits. If the word length of a computer is 32 bits, a single word can store a 32-bit signed number or four ASCII-encoded characters, each occupying 8 bits. A unit of 8 bits is called a *byte*. Machine instructions may require one or more words for their representation. Accessing the memory to store or retrieve a single item of information, either a word or a byte, requires distinct names or *addresses* for each location. It is customary to use numbers from 0 to $2^k - 1$, for some suitable value of k , as the addresses of successive locations in the memory. Thus, the memory can have up to 2^k addressable locations. The 2^k addresses constitute the *address space* of the computer. For example, a 24-bit address generates an address space of 2^{24} (16,777,216) locations. This number is usually written as 16M (16 mega), where 1M is the number 2^{20} (1,048,576).

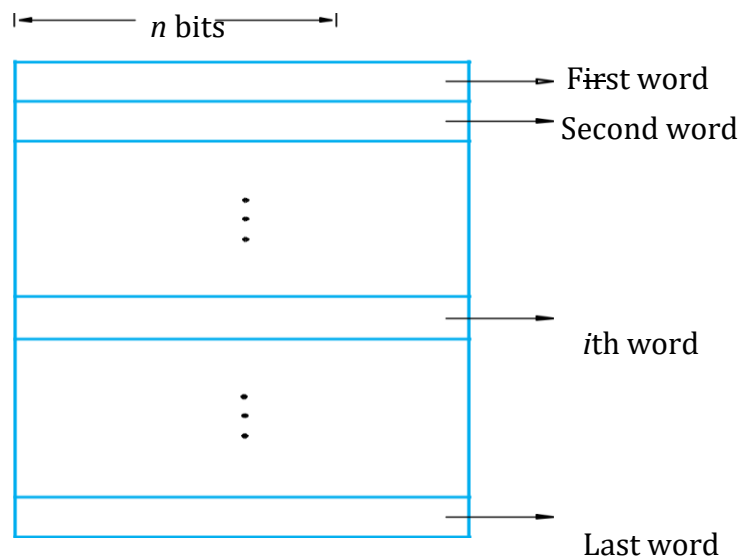


Fig 3.7 Memory words.

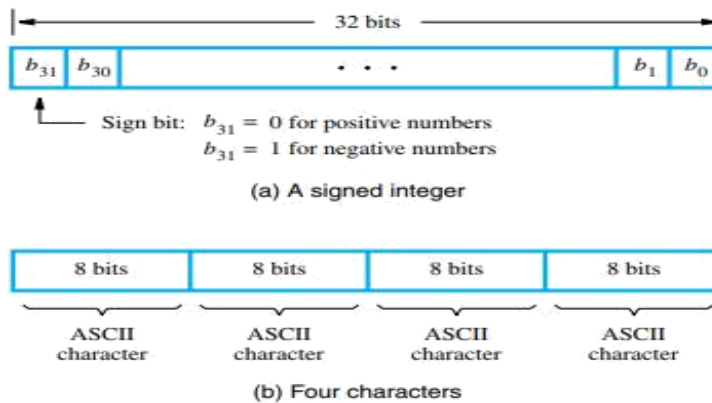


Fig 3.8 Examples of encoded information in a 32-bit word

A 32-bit address creates an address space of 2^{32} or 4G (4 giga) locations, where 1G is 2^{30} . Other notational conventions that are commonly used are K (kilo) for the number 2^{10} (1,024), and T (tera) for the number 2^{40} .

Byte Addressability

The three basic information quantities to deal with: bit, byte, and word. A byte is always 8 bits, but the word length typically ranges from 16 to 64 bits. It is impractical to assign distinct addresses to individual bit locations in the memory. The most practical assignment is to have successive addresses refer to successive byte locations in the memory. This is the assignment used in most modern computers. The term *byte-addressable memory* is used for this assignment. Byte locations have addresses 0, 1, 2, ... Thus, if the word length of the machine is 32 bits, successive words are located at addresses 0, 4, 8, ... with each word consisting of four bytes. There are two ways that byte addresses can be assigned across words, as shown in Figure below. The name *big-endian* is used when lower byte addresses are used for the more significant bytes (the leftmost bytes) of the word. The name *little-endian* is used for the opposite ordering, where the lower byte addresses are used for the less significant bytes (the rightmost bytes) of the word. The words “more significant” and “less significant” are used in relation to the weights (powers of 2) assigned to bits when the word represents a number. Both little-endian and big-endian assignments are used in commercial machines. In both cases, byte addresses 0, 4, 8, ..., are taken as the addresses of

successive words in the memory of a computer with a 32-bit word length. These are the addresses used when accessing the memory to store or retrieve a word.

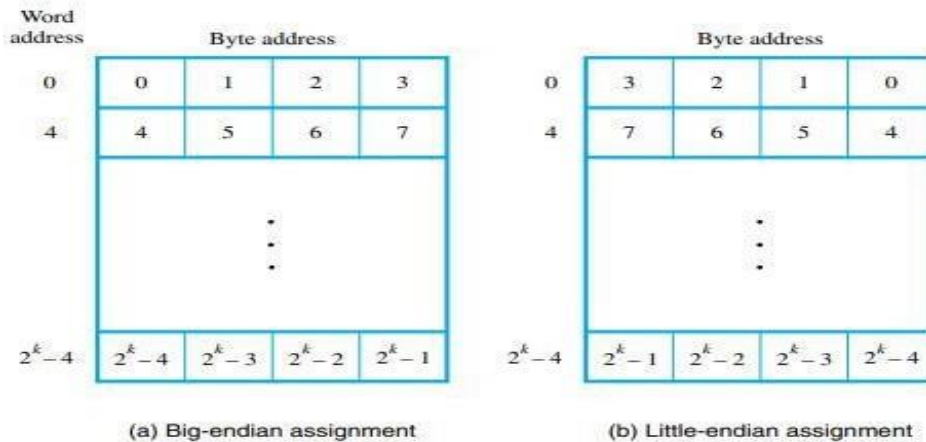


Fig 3.9 Byte and word addressing.

Memory operations

Both program instructions and data operands are stored in the memory. To execute an instruction, the processor control circuits must cause the word (or words) containing the instruction to be transferred from the memory to the processor. Operands and results must also be moved between the memory and the processor. Thus, two basic operations involving the memory are needed, namely, *Read* and *Write*.

The Read operation transfers a copy of the contents of a specific memory location to the processor. The memory contents remain unchanged. To start a Read operation, the processor sends the address of the desired location to the memory and requests that its contents be read. The memory reads the data stored at that address and sends them to the processor.

The Write operation transfers an item of information from the processor to a specific memory location, overwriting the former contents of that location. To initiate a Write operation, the processor sends the address of the desired location to the memory, together with the data to be written into that location. The memory then uses the address and data to perform the write.

❖ **Discuss about the various techniques to represent instruction in a computer system. (April/May 2017)**

The tasks carried out by a computer program consist of a sequence of small steps, such as adding two numbers, testing for a particular condition, reading a character from the keyboard, or sending a character to be displayed on a display screen. A computer must have instructions capable of performing four types of operations:

- Data transfers between the memory and the processor registers
- Arithmetic and logic operations on data
- Program sequencing and control
- I/O transfers

Register Transfer Notation

Predefined names for the processor registers may be R0 or R5. Registers in the I/O subsystem may be identified by names such as DATAIN or OUTSTATUS. To describe the transfer of information, the contents of any location are denoted by placing square brackets around its name. Thus, the expression

$$R2 \leftarrow [LOC]$$

means that the contents of memory location LOC are transferred into processor register R2. As another example, consider the operation that adds the contents of registers R2 and R3, and places their sum into register R4. This action is indicated as

$$R4 \leftarrow [R2] + [R3]$$

This type of notation is known as *Register Transfer Notation* (RTN). Note that the right-hand side of an RTN expression always denotes a value, and the left-hand side is the name of a location where the value is to be placed, overwriting the old contents of that location. In computer jargon, the words “transfer” and “move” are commonly used to mean “copy.” Transferring data from a *source* location A to a *destination* location B means that the contents of location A are read and then written into location B. In this operation, only the contents of the destination will change. The contents of the source will stay the same.

Assembly-Language Notation

Another type of notation to represent machine instructions and programs. For this, we use *assembly language*. For example, a generic instruction that causes the transfer described above, from memory location LOC to processor register R2, is specified by the statement

Load R2, LOC

The contents of LOC are unchanged by the execution of this instruction, but the old contents of register R2 are overwritten. The name Load is appropriate for this instruction, because the contents read from a memory location are *loaded* into a processor register.

The second example of adding two numbers contained in processor registers R2 and R3 and placing their sum in R4 can be specified by the assembly-language statement

Add R4, R2, R3

In this case, registers R2 and R3 hold the source operands, while R4 is the destination.

❖ Explain in detail about instruction sequencing with examples (April/May 2017)

An *instruction* specifies an operation to be performed and the operands involved. In the above examples, we used the English words Load and Add to denote the required operations. In the assembly-language instructions of actual (commercial) processors, such operations are defined by using *mnemonics*, which are typically abbreviations of the words describing the operations. For example, the operation Load may be written as LD, while the operation Store, which transfers a word from a processor register to the memory, may be written as STR or ST. Assembly languages for different processors often use different mnemonics for a given operation.

Executing a given instruction is a two-phase procedure. In the first phase, called *instruction fetch*, the instruction is fetched from the memory location whose address is in the PC. This instruction is placed in the *instruction register* (IR) in the processor. At the start of the second phase, called *instruction execute*, the instruction in IR is examined to determine which operation is to be performed. The specified

operation is then performed by the processor. This involves a small number of steps such as fetching operands from the memory or from processor registers, performing an arithmetic or logic operation, and storing the result in the destination location. At some point during this two-phase procedure, the contents of the PC are advanced to point to the next instruction. When the execute phase of an instruction is completed, the PC contains the address of the next instruction, and a new instruction fetch phase can begin.

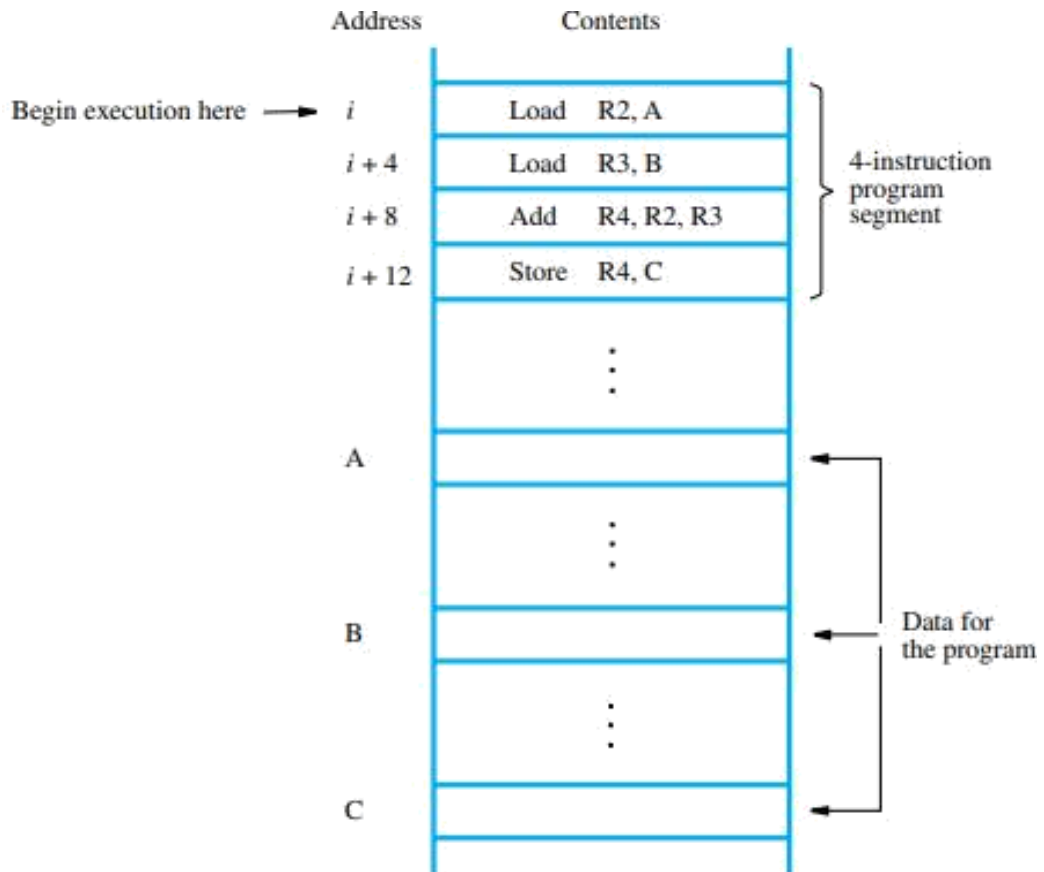


Fig 3.10 A program for $C \leftarrow [A] + [B]$.

BRANCHING

Consider the task of adding a list of n numbers. The addresses of the memory locations containing the n numbers are symbolically given as NUM1, NUM2,..., NUM n , and separate Load and Add instructions are used to add each number to the contents of register R2. After all the numbers have been added, the result is placed in memory location SUM.

Instead of using a long list of Load and Add instructions, it is possible to implement a program loop in which the instructions read the next number in the list and add it to the current sum. To add all numbers, the loop has to be executed as many times as there are numbers in the list. Fig 3.10 shows the structure of the desired program. The body of the loop is a straight-line sequence of instructions executed repeatedly. It starts at location LOOP and ends at the instruction Branch_if_[R2]>0. During each pass through this loop, the address of the next list entry is determined, and that entry is loaded into R5 and added to R3. The address of an operand can be specified in various ways. For now, we concentrate on how to create and control a program loop.

Assume that the number of entries in the list, n , is stored in memory location N, as shown. Register R2 is used as a counter to determine the number of times the loop is executed. Hence, the contents of location N are loaded into register R2 at the beginning of the program. Then, within the body of the loop, the instruction

Subtract R2, R2, #1

reduces the contents of R2 by 1 each time through the loop. Execution of the loop is repeated as long as the contents of R2 are greater than zero. *Branch* instruction loads a new address into the program counter. As a result, the processor fetches and executes the instruction at this new address, called the *branch target*, instead of the instruction at the location that follows the branch instruction in sequential address order.

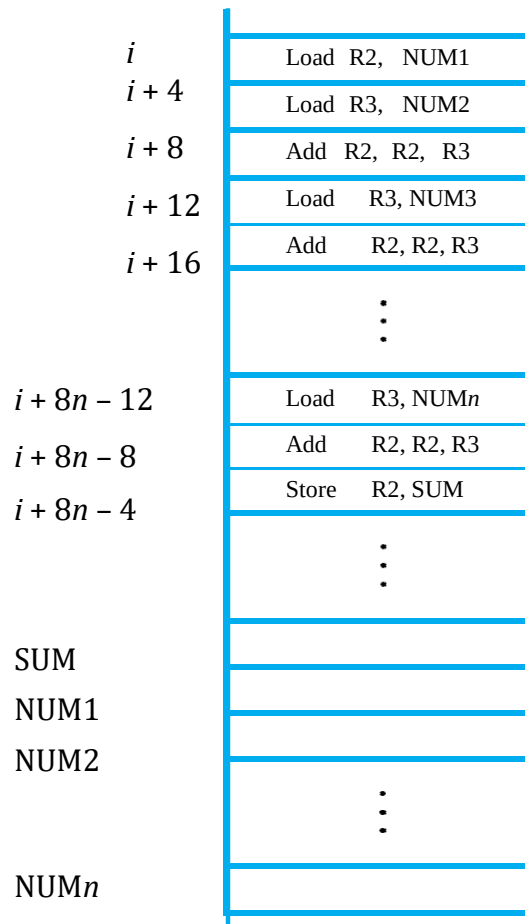


Fig 3.11 A program for adding n numbers.

A *conditional branch* instruction causes a branch only if a specified condition is satisfied. If the condition is not satisfied, the PC is incremented in the normal way, and the next instruction in sequential address order is fetched.

In the program in Fig 3.12, the instruction

Branch_if_[R2]>0 LOOP

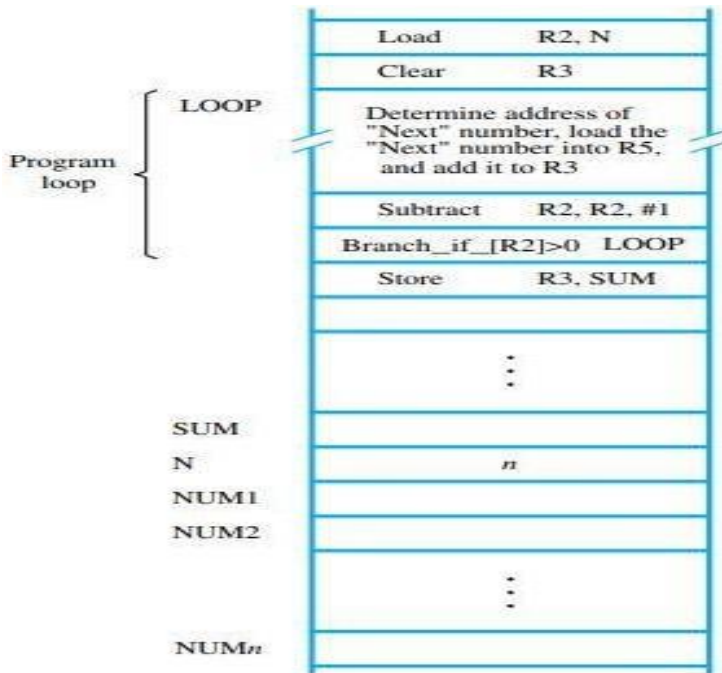


Fig 3.12 Using a loop to add n numbers.

is a conditional branch instruction that causes a branch to location LOOP if the contents of register R2 are greater than zero. This means that the loop is repeated as long as there are entries in the list that are yet to be added to R3. At the end of the n th pass through the loop, the Subtract instruction produces a value of zero in R2, and, hence, branching does not occur. Instead, the Store instruction is fetched and executed. It moves the final result from R3 into memory location SUM. The capability to test conditions and subsequently choose one of a set of alternative ways to continue computation has many more applications than just loop control. Such a capability is found in the instruction sets of all computers and is fundamental to the programming of most nontrivial tasks. One way of implementing conditional branch instructions is to compare the contents of

two registers and then branch to the target instruction if the comparison meets the specified requirement. For example, the instruction that implements the action

Branch_if_[R4]>[R5]

LOOP may be written in generic assembly language as

Branch_greater_than R4, R5, LOOP

or using an actual mnemonic as

BGT R4, R5, LOOP

It compares the contents of registers R4 and R5, without changing the contents of either register. Then, it causes a branch to LOOP if the contents of R4 are greater than the contents of R5.

❖ Explain the operations of stacks and subroutines (April/May 2014)

STACKS

Data operated on by a program can be organized in a variety of ways. A *stack* is a list of data elements, usually words, with the accessing restriction that elements can be added or removed at one end of the list only. This end is called the top of the stack, and the other end is called the bottom. The structure is sometimes referred to as a *pushdown* stack. Another descriptive phrase, *last-in-first-out* (LIFO) stack, is also used to describe this type of storage mechanism; the last data item placed on the stack is the first one removed when retrieval begins. The terms *push* and *pop* are used to describe placing a new item on the stack and removing the top item from the stack, respectively. One processor register, called the *stack pointer* (SP), is used to point to a particular stack structure called the *processor stack*, whose use will be explained shortly.

Data can be stored in a stack with successive elements occupying successive memory locations. Assume that the first element is placed in location BOTTOM, and when new elements are pushed onto the stack, they are placed in successively lower address locations. We use a stack that grows in the direction of decreasing memory addresses in our discussion, because this is a common practice.

Fig 3.13 shows an example of a stack of word data items. The stack contains numerical values, with 43 at the bottom and 28 at the top. The stack pointer, SP, is used to keep track of the address of the element of the

stack that is at the top at any given time. If we assume a byte-addressable memory with a 32-bit word length, the push operation can be implemented as

```
Subtract  SP, SP, #4
Store     Rj, (SP)
```

where the Subtract instruction subtracts 4 from the contents of SP and places the result in SP. Assuming that the new item to be pushed on the stack is in processor register R_j , the Store instruction will place this value on the stack. These two instructions copy the word from R_j onto the top of the stack, decrementing the stack pointer by 4 before the store (push) operation. The pop operation can be implemented as

```
Load      Rj, (SP)
SP, SP,
Add       #4
```

These two instructions load (pop) the top value from the stack into register R_j and then increment the stack pointer by 4 so that it points to the new top element.

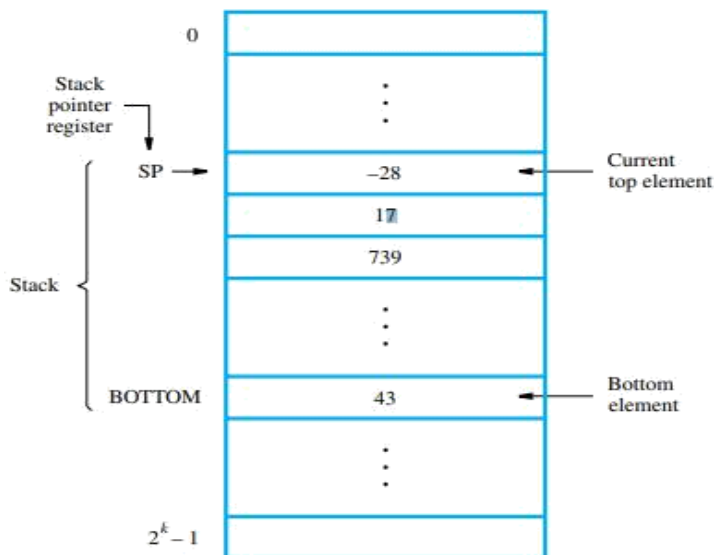


Fig 3.13 A stack of words in the memory.

SUBROUTINES

In a given program, it is often necessary to perform a particular task many times on different data values. It is prudent to implement this task as a block of instructions that is executed each time the task has to be performed. Such a block of instructions is usually called a *subroutine*. For example, a subroutine may evaluate a mathematical function, or it may sort a list of values into increasing or decreasing order. When a program branches to a subroutine we say that it is *calling* the subroutine. The instruction that performs this branch operation is named a Call instruction. After a subroutine has been executed, the calling program must resume execution, continuing immediately after the instruction that called the subroutine. The subroutine is said to *return* to the program that called it, and it does so by executing a Return instruction. Since the subroutine may be called from different places in a calling program, provision must be made for returning to the appropriate location. The location where the calling program resumes execution is the location pointed to by the updated program counter (PC) while the Call instruction is being executed. Hence, the contents of the PC must be saved by the Call instruction to enable correct return to the calling program.

The way in which a computer makes it possible to call and return from subroutines is referred to as its *subroutine linkage* method. The simplest subroutine linkage method is to save the return address in a specific location, which may be a register dedicated to this function. Such a register is called the *link register*. When the subroutine completes its task, the Return instruction returns to the calling program by branching indirectly through the link register.

The Call instruction is just a special branch instruction that performs the following operations:

Store the contents of the PC in the link register
Branch to the target address specified by the Call instruction
The Return instruction is a special branch instruction that performs the operation Branch to the address contained in the link register

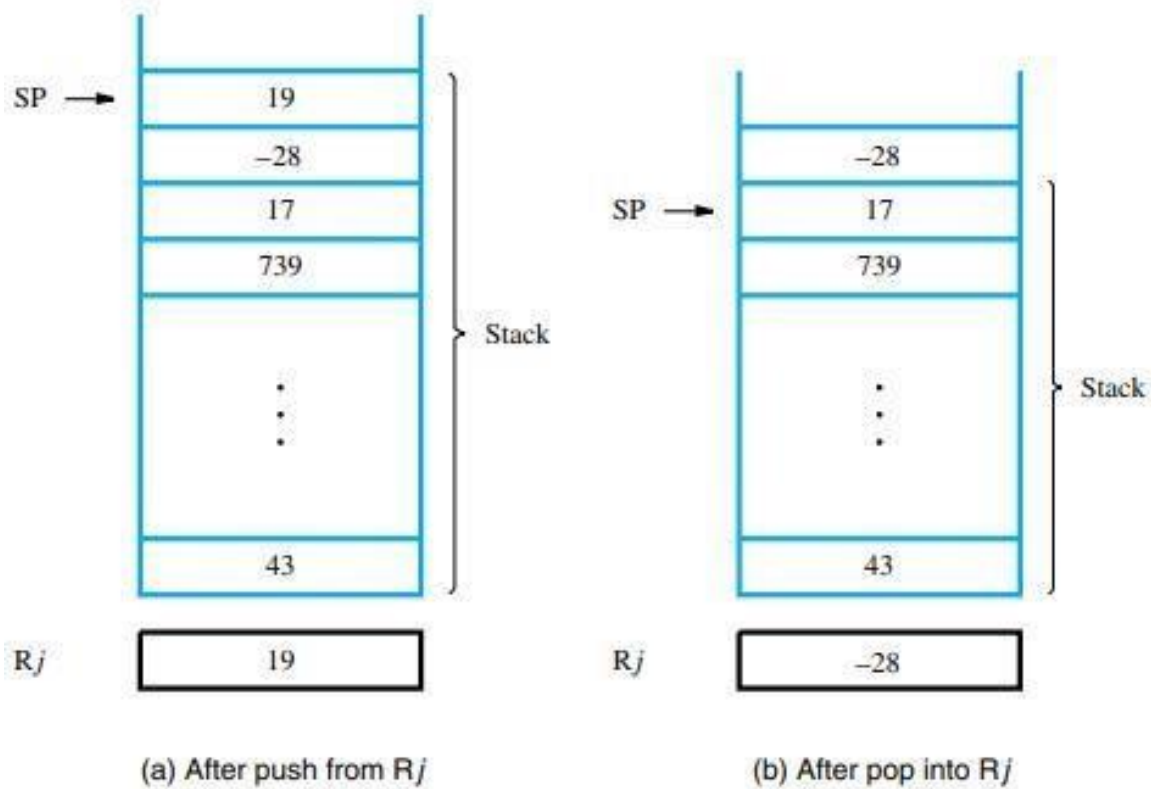


Fig 3.14 Effect of stack operations on the stack in Fig 3.13.

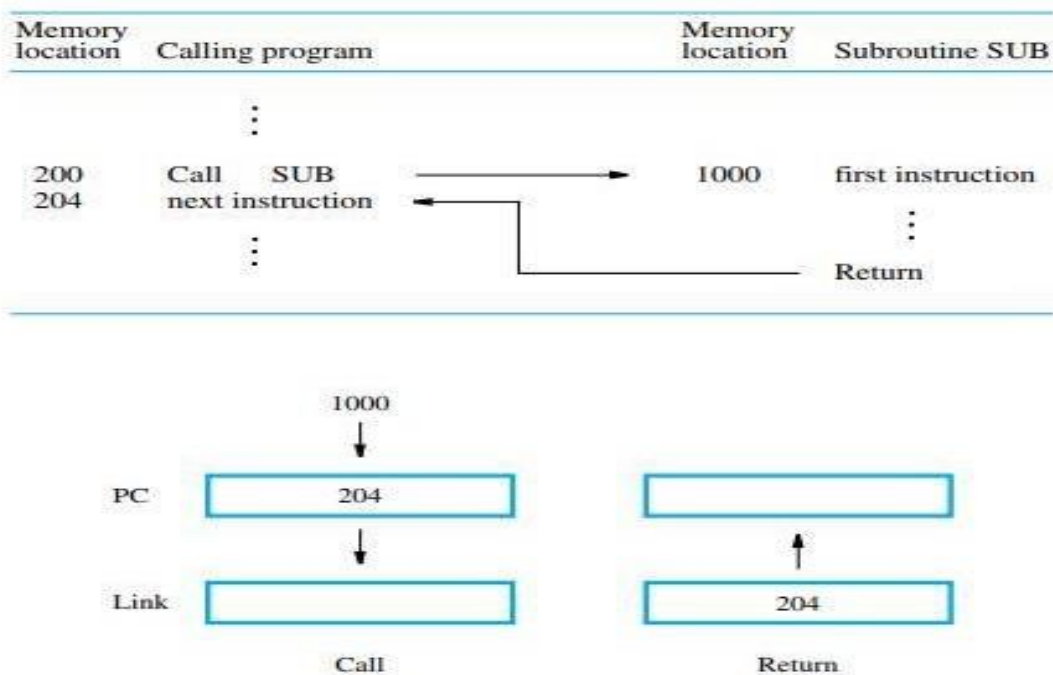


Fig 3.15 Subroutine linkage using a link register.

ADDRESSING AND ADDRESSING MODES

- ❖ What is an addressing mode? Explain the various addressing modes with suitable examples. (April/May 2021) (Nov/Dec 2015).
- ❖ What is the need for addressing in a computer system? Explain the different addressing modes with suitable examples. (April/May 2015).
- ❖ Assume two address format specified as source and destination. Examine the following sequence of instructions and explain the addressing modes used and the operation done in every instruction. (Nov/Dec 2014).

Perform any operation; the corresponding instruction is to be given to the microprocessor.

In each instruction, programmer has to specify 3 things:

- Operation to be performed.
- Address of source of data.
- Address of destination of result.

The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes. The method by which the address of source of data or the address of destination of result is given in the instruction is called Addressing Modes. Computers use addressing mode techniques for the purpose of accommodating one or both of the following provisions:

- To give programming versatility to the user by providing such facilities as pointers to memory, counters for loop control, indexing of data, and program relocation.
- To reduce the number of bits in the addressing field of the instruction.

The MIPS addressing modes are the following:

1. Immediate addressing, where the operand is a constant within the instruction itself
2. Register addressing, where the operand is a register
3. Base or displacement addressing, where the operand is at the memory location whose address is the sum of a register and a constant in the instruction
4. PC-relative addressing, where the branch address is the sum of the PC and a constant in the instruction
5. Pseudo direct addressing, where the jump address is the 26 bits of the instruction concatenated with the upper bits of the PC

Table 3.1 RISC-type addressing modes.

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	R_i	$EA = R_i$
Absolute	LOC	$EA = LOC$
Register indirect	(R_i)	$EA = [R_i]$
Index	$X(R_i)$	$EA = [R_i] + X$
Base with index	(R_i, R_j)	$EA = [R_i] + [R_j]$

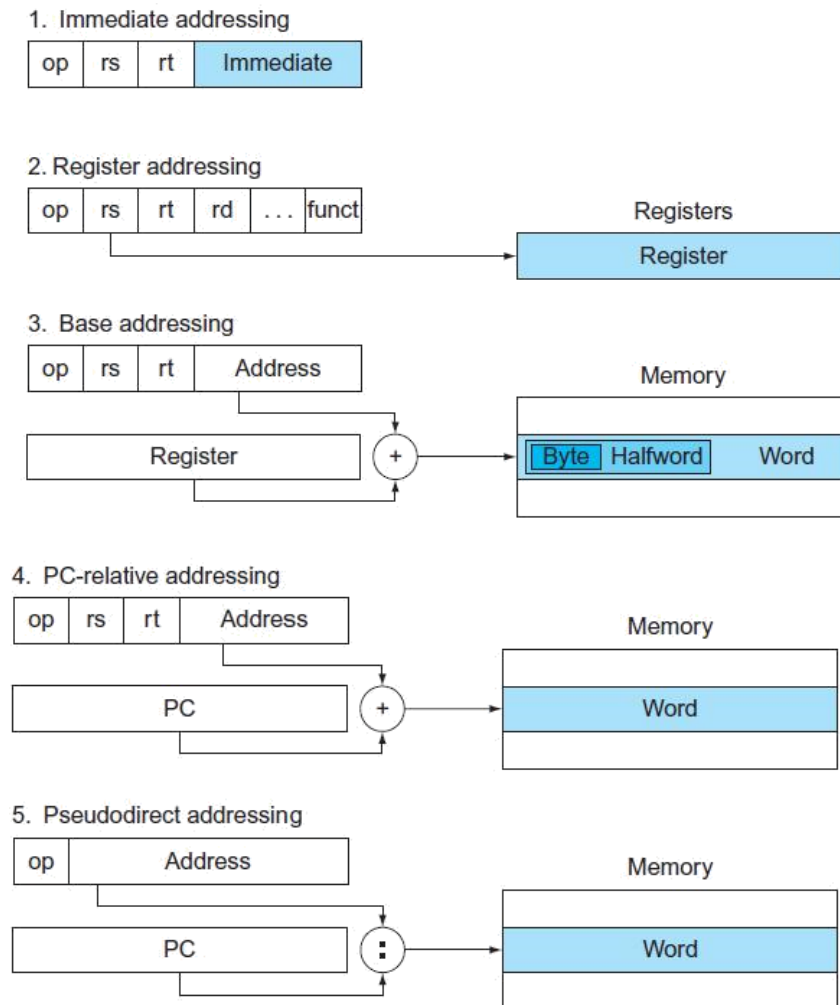


Fig3.16 Various Addressing Modes

TYPES:

- Implied addressing mode
- Immediate addressing mode
- Direct addressing mode
- Indirect addressing mode
- Register addressing mode
- Register Indirect addressing mode
- Auto increment or Auto decrement addressing mode
- Relative addressing mode
- Indexed addressing mode
- Base register addressingmode

a. Implied addressing mode:

In this mode the operands are specified implicitly in the definition of the instruction. For example the 'complement accumulator' instruction is an implied mode instruction because the operand in the accumulator register is implied in the definition of the instruction itself. All register reference instructions that use an accumulator are implied mode instructions. Zero address instructions in a stack organized computer are implied mode instructions since the operands are implied to be on the top of the stack.

b. Immediate addressing mode:

In this mode the operand is specified in the instruction itself. In other words, an immediate mode instruction has an operand field rather than an address field. The operand field contains the actual operand to be used in conjunction with the operation specified in the instruction. Immediate mode instructions are useful for initializing registers to a constant value.

Example: ADD 5

- Add 5 to contents in accumulator

c. Direct addressing mode:

In this mode the effective address is equal to the address part of the instruction. The operand resides in memory and its address is given directly by the address field of instruction. In a branch type instruction the address field specifies the actual branch address.

E.g. LDA A

Look in memory at address A for operand. Load contents of A to accumulator

d. Indirect addressing mode:

In this mode the address field of the instruction gives the address where the effective address is stored in memory/register. Control fetches the instruction from memory and uses its address part to access memory again to read the effective address.

EA = address contained in register/memory

location Example **Add (M)**

- Look in M, find address contained in M and look there for operand
- Add contents of memory location pointed to by contents of M to accumulator

e. Register addressing mode:

In this mode the instruction specifies a register in the CPU whose contents give the effective address of the operand in the memory. In other words, the selected register contains the address of the operand rather than the operand itself. Before using a register indirect mode instruction, the programmer must ensure that the memory address of the operand is placed in the processor register with a previous instruction. The advantage of a register indirect mode instruction is that the address field of the instruction uses fewer bits to select a register than would have been required to specify a memory address directly.

Therefore EA = the address stored in the register R

- Operand is in memory cell pointed to by contents of register R
- Example **Add (R2), R0**

f. Register indirect addressing mode:

In this mode the instruction specifies a register in the CPU whose contents give the effective address of the operand in the memory. In other words, the selected register contains the address of the operand rather than the operand itself. Before using a register indirect mode instruction, the programmer must ensure that the memory address of the operand is placed in the processor register with a previous instruction. The advantage of a register indirect mode instruction is that the address field of the instruction uses fewer bits to select a register than would have been required to specify a memory address directly.

Therefore EA = the address stored in the register R

- Operand is in memory cell pointed to by contents of register R

g. Auto increment or auto decrement addressing mode:

Auto Increment mode:

The effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next item in a list.

We denote the Auto increment mode by putting the specified register in parentheses, to show that the contents of the register are used as the effective address, followed by a plus sign to indicate that these contents are to be incremented after the operand is accessed. Thus, the Auto increment mode is written as **(Ri) +**

The contents of a register specified in the instruction are first automatically decremented and is then used as the effective address of the operand. We denote the Auto decrement mode by putting the specified register in parentheses, preceded by a minus sign to indicate that the contents of the register are to be decremented before being used as the effective address. Thus, we write **-(Ri)**

- These two modes are useful when we want to access a table of data.
ADD (R1)+ will increment the register R1.
LDA -(R1) will decrement the register R1.

h. Relative addressing mode:

In this mode the content of the program counter is added to the address part of the instruction in order to obtain the effective address. Effective address is defined as the memory address obtained from the computation dictated by the given addressing mode. The address part of the instruction is usually a signed number (in 2's complement representation) which can be either positive or negative. When this number is added to the content of the program counter, the result produces an effective address whose position in memory is relative to the address of the next instruction.

Relative addressing is often used with branch type instructions when the branch address is in the area surrounding the instruction word itself. It results in a shorter address field in the instruction format since the relative address can be specified with a smaller number of bits compared to the bits required to designate the entire memory address.

EA = A + contents of PC Example:

PC contains 825 and address part of instruction contains 24.

After the instruction is read from location 825, the PC is incremented to 826. So $EA = 826 + 24 = 850$. The operand will be found at location 850 i.e. 24 memory locations forward from the address of the next instruction.

i. Indexed addressing mode:

In this mode the content of an index register is added to the address part of the instruction to obtain the effective address. The index register is a special CPU register that contains an index value. The address field of the instruction defines the beginning address of a data array in memory. Each operand in the array is stored in memory relative to the beginning address. The distance between the beginning address and the address of the operand is the index value stored in the index register. Any operand in the array can be accessed with the same instruction provided that the index register contains the correct index value. The index register can be incremented to facilitate access to consecutive operands. Note that if an index type instruction does not include an address field in its format, then the instruction converts to the register indirect mode of operation. Therefore $EA = A + IR$

- Example MOV AL, DS: disp [SI]

j. Base register addressing mode:

In this mode the content of base register is added to the address part of the instruction to obtain the effective address. This is similar to the indexed addressing mode except that the register is now called a base register instead of an index register. The

difference between the two modes/is in the way they are used rather than in the way that they are computed.

An index register is assumed to hold an index number that is relative to the address part of the instruction. A base register is assumed to hold a base address and the address field of the instruction gives a displacement relative to this base address. The base register addressing mode is used in computers to facilitate the relocation of the programs in memory. When programs and data are moved from one segment of memory to another, as required in multiprogramming systems, the address values of instructions must reflect this change of position. With a base register, the displacement values of instructions do not have to change.

ENCODING OF MACHINE INSTRUCTIONS

To be executed in a processor, assembly-language instructions must be converted by the assembler program into machine instructions that are encoded in a compact binary pattern.

Let us now examine how machine instructions may be formed.

Add Rdst, Rsrc1, Rsrc2

The Add instruction is representative of a class of three-operand instructions that use operands in processor registers. Registers Rdst, Rsrc1, and Rsrc2 hold the destination and two source operands. If a processor has 32 registers, then it is necessary to use five bits to specify each of the three registers in such instructions. If each instruction is implemented in a 32-bit word, the remaining 17 bits can be used to specify the OP code that indicates the operation to be performed (Figure 3.17a).

	Move	R2, #T	R2 points to string <i>T</i> .
	Move	R3, #P	R3 points to string <i>P</i> .
	Move	R4, N	Get the value <i>n</i> .
	Move	R5, M	Get the value <i>m</i> .
	Subtract	R4, R5	Compute <i>n - m</i> .
	Add	R4, R2	The address of <i>T</i> (<i>n - m</i>).
	Add	R5, R3	The address of <i>P</i> (<i>m</i>).
			Use R6 to scan through
LOOP1:	Move	R6, R2	string <i>T</i> .
			Use R7 to scan through
	Move	R7, R3	string <i>P</i> .

LOOP2:	MoveByte	R8, (R6)+	Compare a pair of
	CompareByte	R8, (R7)+	characters in
	Branch=0	NOMATCH	strings T and P .
	Compare	R5, R7	Check if at $P(m)$.
	Branch > 0	LOOP2	Loop again if not done.
	Move	RESULT, R2	Store the address of $T(i)$.
	Branch	DONE	
			Point to next character in
NOMATCH:	Add	R2, #1	T .
	Compare	R4, R2	Check if at $T(n - m)$.
	Branch ≥ 0	LOOP1	Loop again if not done.
	Move	RESULT, # -1	No match was found.
	next		
DONE:	instruction		

Figure 3.17 A CISC-style program for string search.

Now consider instructions in which one operand is given using the Immediate addressing mode, such as

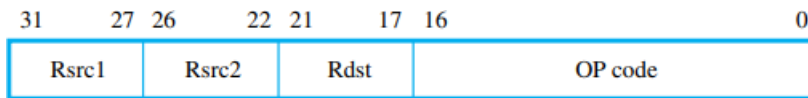
Add Rdst, Rsrc, #Value

Of the 32 bits available, ten bits are needed to specify the two registers. The remaining 22 bits must give the OP code and the value of the immediate operand. The most useful sizes of immediate operands are 32, 16, and 8 bits. Since 32 bits are not available, a good choice is to allocate 16 bits for the immediate operand. This leaves six bits for specifying the OP code. This format can also be used for Load and Store instructions, where the Index addressing mode uses the 16-bit field to specify the offset that is added to the contents of the index register.

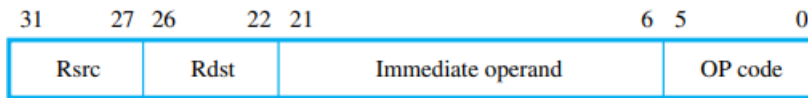
The format in Figure 3.18b can also be used to encode the Branch instructions. Consider the program in Figure 3.17. The Branch-greater-than instruction at memory address 128 could be written in a specific assembly language as

BGT R2, R0, LOOP

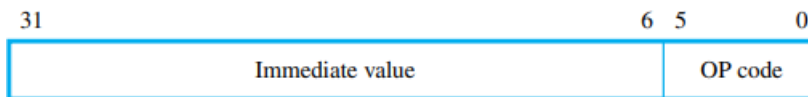
if the contents of register R0 are zero. The registers R2 and R0 can be specified in the two register fields in Figure 3.18. The six-bit OP code has to identify the BGT operation. The 16-bit immediate field can be used to provide the information needed to determine the branch target address, which is the location of the instruction with the label LOOP.



(a) Register-operand format



(b) Immediate-operand format



(c) Call format

Figure 3.18 Possible instruction formats.

The target address generally comprises 32 bits. Since there is no space for 32 bits, the BGT instruction makes use of the immediate field to give an offset from the location of this instruction in the program to the required branch target. At the time the BGT instruction is being executed, the program counter, PC, has been incremented to point to the next instruction, which is the Store instruction at address 132. Therefore, the branch offset is $132 - 112 = 20$. Since the processor computes the target address by adding the current contents of the PC and the branch offset, the required offset in this example is negative, namely -20 .

Finally, we should consider the Call instruction, which is used to call a subroutine. It only needs to specify the OP code and an immediate value that is used to determine the address of the first instruction in the subroutine. If six bits are used for the OP code, then the remaining 26 bits can be used to denote the immediate value. This gives the format shown in Figure 3.18c.

INTERACTION BETWEEN ASSEMBLY AND HIGH LEVEL LANGUAGE

The compiler translates a simple high level language program to assembly code. After the compiler generates the assembly level code, the assembler generates the machine level code. This assembler generates something called as opcode, this opcode is like instructions for the CPU. The opcode will have a format and the

predefined address space will be embedded in the opcode. Two example programs will be discussed in this section.

VECTOR DOT PRODUCT PROGRAM

The first example is a numerical application that is an extension of previous programs for adding numbers. In calculations that involve vectors and matrices, it is often necessary to compute the dot product of two vectors. Let A and B be two vectors of length n . Their dot product is defined as

$$\text{Dot Product} = \sum_{i=0}^{n-1} A(i) \times B(i)$$

Figures 3.19 shows RISC-style program for computing the dot product and storing it in memory location DOTPROD. The first elements of each vector, $A(0)$ and $B(0)$, are stored at memory locations AVEC and BVEC, with the remaining elements in the following word locations. The task of accumulating a sum of products occurs in many signal-processing applications. In this case, one of the vectors consists of the most recent n signal samples in a continuing time sequence of inputs to a signal-processing unit. The other vector is a set of n weights. The n signal samples are multiplied by the weights, and the sum of these products constitutes an output signal sample. Some computer instruction sets combine the operations of the Multiply and Add instructions used in the programs in Figures 3.19 and 3.20 into a single Multiply Accumulate instruction. This is done in the processor.

	Move	R2, #AVEC	R2 points to vector A.
	Move	R3, #BVEC	R3 points to vector B.
	Load	R4, N	R4 serves as a counter.
	Clear	R5	R5 accumulates the dot product.
LOOP:	Load	R6, (R2)	Get next element of vector A.
	Load	R7, (R3)	Get next element of vector B.
	Multipl y	R8, R6, R7	Compute the product of next pair.
	Add	R5, R5, R8	Add to previous sum.
	Add	R2, R2, #4	Increment pointer to vector A.
	Add	R3, R3, #4	Increment pointer to vector B.
	Subtra ct	R4, R4, #1	Decrement the counter.
	Branch_if_ >0	[R4] LOOP	Loop again if not done.
	Store	R5, DOTPROD	Store dot product in memory.

Figure 3.19 A RISC-style program for computing the dot product of two vectors.

STRING SEARCH PROGRAM

As an example of a non-numerical application, let us consider the problem of string search. Given two strings of ASCII-encoded characters, a long string T and a short string P , we want to determine if the pattern P is contained in the target T . Since P may be found in T in several places, we will simplify our task by being interested only in the first occurrence of P in T when T is searched from left to right. Let T and P consist of n and m characters, respectively, where $n > m$. The characters are stored in memory in consecutive byte locations. Assume that the required data are located as follows:

6. T is the address of $T(0)$, which is the first character in string T .
7. N is the address of a 32-bit word that contains the value n .
8. P is the address of $P(0)$, which is the first character in string P .
9. M is the address of a 32-bit word that contains the value m .
10. RESULT is the address of a word in which the result of the search is to be stored. If the substring P is found in T , then the address of the corresponding location in T will be stored in RESULT; otherwise, the value -1 will be stored.

```

for  $i \leftarrow 0$  to  $n - m$  do
     $b \leftarrow 0$ 
    while  $j < m$  and  $P[j] = T[i + j]$  do
         $j \leftarrow j + 1$ 
    if  $j = m$  return  $i$ 
return  $-1$ 

```

Figure 3.37 A brute-force string search algorithm.

	Move	R2, #T	R2 points to string T .
	Move	R3, #P	R3 points to string P .
	Load	R4, N	Get the value n .
	Load	R5, M	Get the value m .
	Subtract	R4, R4, R5	Compute $n - m$.
	Address	R4, R2, R4R5,	The address of $T(n - m)$. The address of
LOOP1:	Address	R3, R5	$P(m)$.
		R6, R2	Use R6 to scan through string T .
	Move		.
	Move	R7, R3	Use R7 to scan through string P .
LOOP2:	LoadByte	R8, (R6)	Compare a pair of characters in
	LoadByte	R9, (R7)	strings T and P .
	Branch_if_[R8]=[R9]	NOMATCH	Point to next character in T .
	Add	CHR6,	Point to next character in T .
	Add	R6, #1	Point to next character in P .
	Branch_if_[R5]>[R7]	R7, R7, #1	Loop again if not done.
	Store	LOOP2	
		R2, RESULT	Store the address of $T(i)$.
	Branch	DONE	
NOMATCH:	hAdd	R2, R2, #1	Point to next character in T .
	Branch_if_[R4]≥[R2]	LOOP	Loop again if not done.
	Move	1 R8,	Write -1 to indicate that no match was found.
	Store	# -1	
		R8, RESULT	
DONE:	next instruction		

Figure 3.20 A RISC-style program for string search.

String search is an important and well-researched problem. Many algorithms have been developed. Since our main purpose is to illustrate the use of assembly-language instructions, we will use the simplest algorithm which is known as the *brute-force* algorithm. It is given in Figure 3.19. In a RISC-style computer, the algorithm can be implemented as shown in Figure 3.20. The comments explain the use of various processor registers. Note that in the case of a failed search, the immediate value -1 will cause the contents of R8 to become equal to 0xFFFFFFFF, which represents -1 in 2's complement.

Two Marks Questions and Answers:

1. What are the basic functional units of a computer?

Ans: A computer consists of five functionally independent main parts

namely -Input Unit

-Memory Unit

-Arithmetic and logic Unit

-Output Unit

-Control Unit

2. Define RAM.

Ans: Memory in which any location can be reached in a short and fixed amount of time after specifying its address is called random access memory.

3. Define memory access time.

Ans: The time required to access one word is called memory access time.

4. What is instruction register (IR) and program counter (PC) used for

? Ans: The instruction register (IR) holds the instruction that is currently being executed. Its output is available to the control circuits which generate the timing signals that control the various processing elements. The program counter (PC) is used to keep track of the execution of the program. It contains the memory address of the next instruction to be fetched and executed.

5. What do you mean by memory address register (MAR) and memory data register (MDR)?

The MAR holds the address of the location to be accessed. The MDR contains

6. What is an interrupt?

Ans: An interrupt is a request from an I/O device for service by the processor. The processor provides the requested service by executing an appropriate interrupt service routine.

7. Explain about Bus.

Ans: Bus is a group of lines that serves as a connecting path for several devices. In addition to the lines that carry the data, the bus must have the lines for address and control purposes.

8. What do you mean by multiprogramming or multitasking?

Ans: The operating system manages the concurrent execution of several application programs to make best possible use of computer resources. This pattern of concurrent execution is called multiprogramming or multitasking.

9. What is Big-Endian and Little-Endian representations.

Ans: The Big-endian is used when lower byte addresses are used for the more significant bytes (The leftmost bytes) of the word. The little-endian is used for the opposite ordering, where the lower byte addresses are used for the less significant bytes (the rightmost bytes) of the word.

10. What are the different types of addressing modes available?

Ans: The different types of addressing modes available are: -Immediate addressing mode

-Register addressing mode

-Direct or absolute addressing mode

-Indirect addressing mode

-Indexed addressing mode

mode

-Relative addressing mode

-Auto increment

-Auto decrement

11. What is indirect addressing mode?

Ans: The effective address of the operand is the contents of a register or memory location whose address appears in the instruction

12. What is indexed addressing mode?

Ans: The effective address of the operand is generated by adding a constant value to the contents of a register.

13. Define autoincrement mode of addressing?

Ans: The effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next item in the list.

14. Define autodecrement mode of addressing?

Ans: The contents of a register specified in the instruction are first automatically decremented and are then used as the effective address of the operand.

15. What are condition code flags? What are the commonly used flags?

Ans: The processor has to keep track of the information about the results of various operations for the

subsequent conditional branch instructions. This is done by recording required information in individual bits called condition code flags.

Four commonly used flags are:

-N(Negative)

-Z(Zero)

-V(overflow)

-C(Carry)

16. What do you mean by assembler directives?

Ans: These are the instructions which direct the program to be executed. They have no binary equivalent so they are called pseudo-opcodes. These instructions are used to define symbols, allocate space for variable, generate fixed tables etc. Examples : END, NAME

17. What do you mean by relative addressing mode?

Ans: The effective address is determined by the index mode using the program counter in place of the general purpose register Ri.

18. What is Stack?

Ans: A stack is a list of data elements, usually words or bytes with the accessing restriction that elements can be added or removed at one end of the list only. It follows last in first out (LIFO) mechanism.

19. What is a queue?

Ans: Is a type of datastructure in which the data are stored in and retrieved on a First in first out(FIFO) basis. It grows in the direction of increasing addresses in the memory. New data are added at the back (High-address end) and retrieved from the front (low-address end) of the queue.

20. What is a pointer?

The register or memory location that contains the address of an operand is called a pointer.

21. What is index register?

In index mode the effective address of the operand is generated by adding a constant value to the contents of a register. The register used may be either a special register or may be any one of a set of general purpose registers in the processor. This register is referred to as an index register.

22. What is assembly language?

A complete set of symbolic names and rules for the use of machines constitute a programming language, generally referred to as an assembly language.

23. Define addressing mode.

The different ways in which the location of an operand is specified in an instruction are referred to as addressing modes.

24. What is byte addressable memory?

The assignment of successive addresses to successive byte locations in the memory is called byte addressable memory.

25. What is big endian and little endian format?

The name big endian is used when lower byte addresses are used for the more significant of the word. The name little endian is used for the less significant bytes of the word.

26. What is a branch instruction?

Branch instruction is a type of instruction which loads a new value into the program counter.

27. What is the function of ALU?

Most of the computer operations (arithmetic and logic) are performed in ALU. The data required for the operation is brought by the processor and the operation is performed by the ALU.

28. What is the function of CU?

The control unit acts as the nerve center, that coordinates all the computer operations. It issues timing signals that governs the data transfer.

29. What are basic operations of a

computer? The basic operations are READ and WRITE.

30. What are the registers generally contained in the processor?

MAR-Memory Address Register
MDR-Memory Data Register
IR-Instruction Register
R0-Rn-General purpose Registers
PC-Program Counter

31. Define Computer Architecture

Computer Architecture is defined as the functional operation of the individual H/W unit in a computer system and the flow of information among the control of those units.

32. What are the functions of control unit ?

The memory arithmetic and logic, and input and output units store and process information and perform i/p and o/p operation, the operation of these units must be coordinated in some way. This is the task of the control unit. The CU is effectively the nerve center that sends the control signal to other units and senses their states.

33. What is an interrupt?

An interrupt is an event that causes the execution of one program to be suspended and another program to be executed.

34. What are the uses of interrupts?

- Recovery from errors
- Debugging
- Communication between programs

35. What is absolute addressing mode?

The address of the location of the operand is given explicitly as a part of the instruction. Eg. Move a, 2000

36. Specify three types of data transfer techniques.

- Arithmetic data transfer
- Logical data transfer
- Programmed control data transfer

37. What is the role of MAR and MDR?

The MAR (memory address register) is used to hold the address of the location to or from which data are to be transferred and the MDR (memory data register) contains the data to be written into or read out of the addressed location.

38. What are the various types of operations required for instructions?

- Data transfers between the main memory and the CPU registers
- Arithmetic and logic operation on data
- Program sequencing and control
- I/O transfers